

1. Access Control (AC)

Its function is to control which principals (persons, processes, machines, ...) have access to which resources in the system -- which files they can read, which programs they can execute, and how they share data with other principals, and so on.

- **Explanation:** Access control is the mechanism that dictates which users, processes, or machines (principals) can interact with specific resources within a system. This includes actions like reading files, executing programs, and sharing data. It's a fundamental aspect of information security at every level from hardware to applications.
- **Examples:**
 - A user logging into their bank account and being able to view their account balance but not the balances of other users.
 - A program only being allowed to read certain system configuration files and not allowed to modify them.
- **Exercises:**
 1. Think of a scenario in a hospital system where different user roles (doctor, nurse, admin) have varying access rights to patient records. How would you design the access control system?
 2. Consider a cloud storage service. What kind of access controls would be necessary to ensure that only authorized users can access their stored files and to protect against unauthorized access?

2. Access Control Matrix (ACM)

- Subjects $S = \{s_1, \dots, s_n\}$
- Objects $O = \{o_1, \dots, o_m\}$
- Rights $R = \{r_1, \dots, r_k\}$.
- Entries $A[s_i, o_j] \subseteq R$ and $A[s_i, o_j] = \{r_x, \dots, r_y\}$ means subject s_i has rights $r_x, \dots, r_y$ over object o_j
- **Explanation:** The Access Control Matrix is a model used to represent the protection state of a system. It's essentially a table where rows represent subjects (active entities such as users or processes), columns represent objects (passive resources like files or printers), and the cells at each intersection define the rights (e.g., read, write, execute) that a subject has over an object.

- **Example:**

- Processes p, q
- Files f, g
- Rights r, w, x, a, o

	f	g	p	q
p	rwo	r	$rwxo$	w
q	a	ro	r	$rwxo$

- **Example:**

- Procedures $inc_ctr, dec_ctr, manage$
- Variable $counter$
- Rights $+, -, call$

	$counter$	inc_ctr	dec_ctr	$manage$
inc_ctr	$+$			
dec_ctr	$-$			
$manage$	$call$	$call$	$call$	

- **Examples:**

- A matrix where a row is a user, a column is a file, and the cell has the read right if the user can read that file.
- A matrix where processes are subjects and procedures are objects, and cells have the 'call' right if the process can call that procedure.

- **Exercises:**

1. Draw an access matrix for a simple system involving three users (Alice, Bob, Carol) and two files (file1, file2). Assign different read, write, and execute rights to each user for each file.
2. How would an access matrix represent a system where a user can only view certain parts of a database and not others? How would you represent that in the matrix?

2.2. Implementations of ACM

2.2.1. Access Control Lists (ACLs)

- **Explanation:** An ACL is a list of permissions attached to an object that specifies which subjects can perform what actions on it. Each object has its own ACL, making it object-centric.
- **Examples**
 - A file has an ACL that says “User Joe can read, write, and own the file” and “User Sam can read and write the file”.

File 1	File 2
Joe: Read	Joe: Read
Joe: Write	Sam: Read
Joe: Own	Sam: Write
	Sam: Own

- A printer may have an ACL that specifies which user or group can use it.
- **Exercises:**
 1. How would you create an ACL for a shared calendar where one user can modify events and other users can only view them?
 2. Given an object and an ACL, how would you determine if a particular subject is allowed to perform a particular operation?

2.2.2. Capability Lists

- **Explanation:** A capability list is a user-centric approach where each user (subject) has a list of access rights they hold over various objects. The list contains all the access rights for one user.
- **Examples:**
 - A user has a capability list that indicates they have read, write, and execute permissions to a specific file and read-only access to another file:
Fred → /dev/console(RW) → fred/prog.c(RW) → fred/letter(RW) → /usr/ucb/vi(X)
Jane → /dev/console(RW) → fred/prog.c(R) → fred/letter() → /usr/ucb/vi(X)

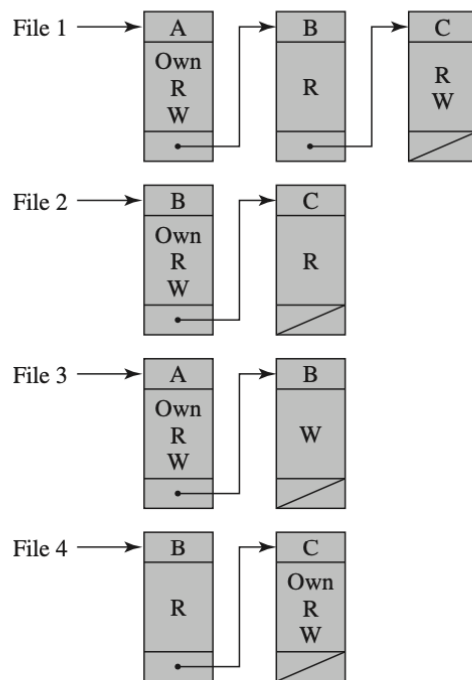
- A program can only access the data it needs through the set of capabilities it has been granted as a result of some communication.

- **Exercises:**

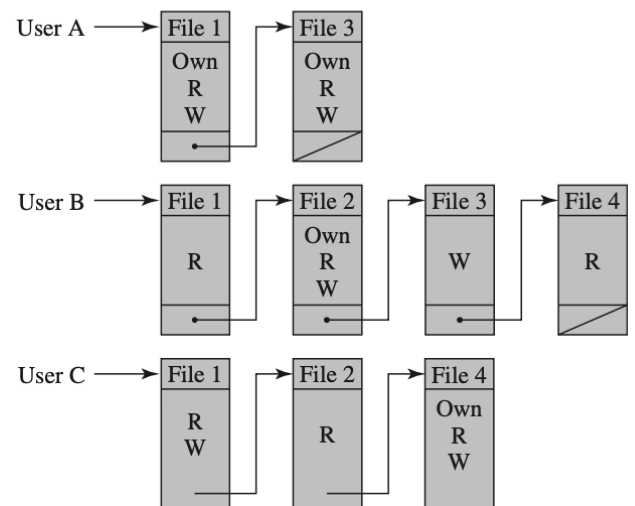
1. How would capability lists need to be managed in a system to ensure that users only have the necessary capabilities?
2. Compare and contrast how granting and revoking access rights are handled in a capability-based system versus an ACL based system.

		OBJECTS			
		File 1	File 2	File 3	File 4
SUBJECTS	User A	Own Read Write		Own Read Write	
	User B	Read	Own Read Write	Write	Read
	User C	Read Write	Read		Own Read Write

(a) Access matrix



(b) Access control lists for files of part (a)



(c) Capability lists for files of part (a)

Figure 4.2 Example of Access Control Structures

2.2.3. Access Control Triples

- **Explanation:** Access control triples represent access control information in a simple format that shows the subject, the access they have and the object that the access applies to.
- **Example:**

Subject	Access	Object
Joe	Read	File 1
Joe	Write	File 1
Joe	Own	File 1
Joe	Read	File 2
Sam	Read	File 2
Sam	Write	File 2
Sam	Own	File 2

- “Joe Read File 1” means that user Joe has read access to File 1.
 - “Sam Write File 2” means that user Sam has write access to File 2.
- **Exercises:**
 1. Describe a simple access control scenario using access control triples.
 2. How can you derive the same information from an access control matrix?

2.3. UNIX Access Control

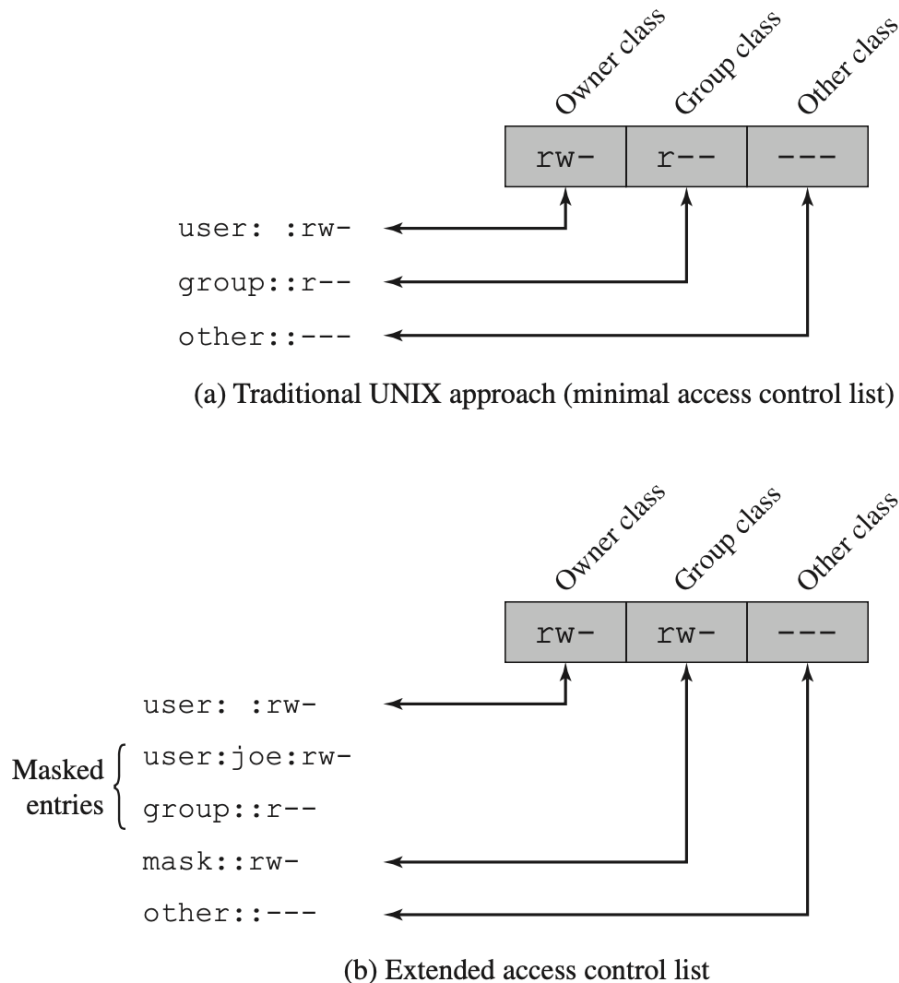


Figure 4.5 UNIX File Access Control

FreeBSD ACL (Access Control List)

FreeBSD allows the administrator to assign a list of UNIX user IDs and groups to a file by using the `setfacl` command. Any number of users and groups can be associated with a file, each with three protection bits (read, write, execute), offering a flexible mechanism for assigning access rights. A file need not have an ACL but may be protected solely by the traditional UNIX file access mechanism. FreeBSD files include an additional protection bit that indicates whether the file has an extended ACL.

FreeBSD and most UNIX implementations that support extended ACLs use the following strategy (e.g., Figure 4.5b):

2. **Owner Class and Other Class Entries**

The owner class and other class entries in the 9-bit permission field have the same meaning as in the minimal ACL case.

3. **Group Class Entry**

The group class entry specifies the permissions for the owner group for this file. These permissions represent the maximum permissions that can be assigned to named users or named groups, other than the owning user. In this latter role, the group class entry functions as a mask.

4. **Additional Named Users and Named Groups**

Additional named users and named groups may be associated with the file, each with a 3-bit permission field. The permissions listed for a named user or named group are compared to the mask field. Any permission for the named user or named group that is not present in the mask field is disallowed.

File Access Request Process

When a process requests access to a file system object, two steps are performed:

5. **Step 1: Select Matching ACL Entry**

The ACL entry that most closely matches the requesting process is selected. The ACL entries are checked in the following order:

- Owner
- Named Users
- (Owning or Named) Groups
- Others

Only a single entry determines access.

6. **Step 2: Check Permissions**

- If the matching entry contains sufficient permissions, access is granted.
- A process can be a member of more than one group; thus, more than one group entry can match.
- If any matching group entry contains the requested permissions, the access is granted.
- If none of the matching group entries contains the requested permissions, access will be denied.

3. Discretionary Access Control (DAC)

1. Explanation of DAC: Discretionary Access Control (DAC) is a type of access control policy where the owner or creator of a resource (like a file or folder) has the discretion to decide who can access it and what actions they can perform (e.g., read, write, execute). The control is based on the identity of the requestor and specific permissions set by the resource owner.

This policy is called “discretionary” because: - The owner has the authority to grant or revoke access. - The owner can transfer access rights to others, enabling them to further grant access.

2. Examples:

Example 1: File Permissions in Unix/Linux

In Unix/Linux systems, file permissions are a classic example of DAC.

- **Scenario:** User Alice creates a file named `project.txt`.
- **Permissions:** By default, Alice is the owner of the file. She can set permissions using the `chmod` command to specify:
 - Who can read, write, or execute the file (user, group, others).
- **Granting Access:** Alice uses:

```
chmod 764 project.txt
```

 - 7: Owner (Alice) can read, write, and execute.
 - 6: Group members can read and write.
 - 4: Others can only read.
- Alice can also explicitly share the file with another user, say Bob, by adding him to the group or using Access Control Lists (ACLs).

Example 2: Windows File Sharing

In Windows systems, file and folder permissions also implement DAC.

- **Scenario:** User John creates a folder named Reports on his computer.
- **Permissions:** John can:
 - Allow his colleague Mary to view (read) the folder but not modify it.
 - Grant Mike full control over the folder, enabling him to read, modify, or delete files.
- **Granting Access:** John right-clicks the folder, goes to **Properties > Security**, and assigns specific permissions to Mary and Mike.

Example 3: Database Access Control

Databases often use DAC to control user privileges.

- **Scenario:** A database administrator (Admin) creates a table named EmployeeData.
- **Permissions:** Admin grants:
 - SELECT permission to UserA to read the data.
 - INSERT and UPDATE permissions to UserB to modify the data.
- **Discretionary Nature:** If UserB has GRANT OPTION privilege, they can further grant INSERT or UPDATE permissions to another user, like UserC.

Key Characteristics of DAC:

1. **Flexibility:** Resource owners have full control over who accesses their resources and what they can do.
2. **Transitivity:** The owner can delegate access, and those granted access might further delegate it (if allowed).
3. **Identity-Based:** Access decisions are based on the identity of the requestor and specific rules defined by the owner.

Advantages and Disadvantages of DAC:

Advantages:

- Simple to understand and implement.
- Highly flexible for resource owners.

Disadvantages:

- Less secure in environments requiring strict control because delegated users might misuse permissions.
- Susceptible to malicious privilege escalation.

DAC is widely used in systems where flexibility is more important than strict control. For systems requiring stronger, centralized policies, Mandatory Access Control (MAC) or Role-Based Access Control (RBAC) might be preferred.

4. Mandatory Access Control (MAC)

Definition: Mandatory Access Control (MAC) is a type of access control policy in which access to resources is determined by a **central authority** based on predefined security rules. The resource owner does not have the authority to decide who gets access.

Key Characteristics:

1. **Centralized Policy Enforcement:**

- Policies are defined by a governing authority, not by the resource owner.
- Users cannot modify access rights.

2. **Label-Based Control:**

- Each resource and user is assigned a **security label** (e.g., “Confidential,” “Secret”).
- Access is granted based on matching these labels.

3. **Non-Discretionary:**

- The system strictly enforces rules without allowing delegation or user-driven changes.

Examples

Example 1: Security Clearance in Government Systems

- In government or military systems:
 - **Resources** (e.g., documents) are labeled with security levels like Top Secret, Secret, Confidential.
 - **Users** are assigned clearance levels.
 - **Rule:** A user can access a document only if their clearance level matches or exceeds the document’s classification.
 - For example:
 - User Alice has a clearance level of Secret.
 - She can access documents labeled Secret or Confidential but **not** Top Secret.

Example 2: Healthcare Information Systems

- In a hospital:
 - Medical records are labeled with confidentiality levels (e.g., Doctor Access, Nurse Access, Admin Access).
 - **Rule:**
 - Doctors can view all patient records.
 - Nurses can only view records for patients under their care.
 - Admin staff can access billing details but not medical details.

For example: A nurse cannot view records of a patient outside their care, even if they know the patient's name.

Exercises

Exercise 1: Document Classification

- **Scenario:** A university uses MAC to manage access to its research files:
 - Research papers are labeled Public, Restricted, and Highly Restricted.
 - Staff and students are assigned access levels:
 - **Staff:** Can access Public and Restricted.
 - **PhD Students:** Can access Public.
 - **Professor:** Can access all levels.
- **Task:**
 1. Determine if a PhD student can access a Restricted research paper.
 2. Identify who can access a Highly Restricted paper.

Exercise 2: Healthcare Labels

- **Scenario:** A hospital uses MAC to control access to medical records:
 - Records are labeled General Access, Doctor Only, and Specialist Only.
 - Users include:
 - **Nurse:** Access to General Access.
 - **Doctor:** Access to General Access and Doctor Only.
 - **Specialist Doctor:** Access to all levels.
- **Task:**
 1. Can a nurse view Doctor Only records?
 2. If a doctor tries to access a Specialist Only record, will they succeed?
 3. Explain why or why not for both cases.

Summary

MAC is highly secure and suitable for environments requiring strict control, such as government and healthcare. However, it lacks flexibility, making it less ideal for systems requiring user-driven discretion, like collaborative work environments.

5. Bell-LaPadula (BLP) Model

Definition: The Bell-LaPadula Model is a formal security model used primarily to maintain **confidentiality** of information in systems that handle classified data. It ensures that information flows only in ways that prevent unauthorized access to higher classification levels.

Key Principles:

7. **Simple Security Property** (“No Read Up” or **SS Property**):
 - A subject at a given security level **cannot read data** at a higher security level.
 - Example: A user with Confidential clearance cannot read data classified as Secret.
8. **Star Property (★)** (“No Write Down” or ***-Property**):
 - A subject at a given security level **cannot write data** to a lower security level.
 - Example: A user with Secret clearance cannot write data to a Confidential file to prevent information leakage.
9. **Discretionary Security Property:**
 - Access is also controlled by DAC mechanisms within the system.

Examples

Example 1: Military Document Management

- **Scenario:**
 - Documents are classified as Top Secret, Secret, and Confidential.
 - Users have clearance levels based on their roles:
 - **General:** Top Secret
 - **Colonel:** Secret
 - **Captain:** Confidential
- **Rules:**
 1. A Captain **cannot read** a Secret or Top Secret document (**No Read Up**).
 2. A Colonel **cannot write** Secret information into a Confidential document (**No Write Down**).

Example 2: Secure Financial Systems

- **Scenario:**
 - A financial system classifies transaction data:
 - **Public:** Open to everyone.
 - **Sensitive:** Accessible only to managers.
 - **Highly Sensitive:** Accessible only to directors.
- **Rules:**
 3. A manager **cannot access** Highly Sensitive data (**No Read Up**).

4. A director **cannot save** Highly Sensitive data in a location marked Public (No Write Down).

Exercises

Exercise 1: Applying Security Properties

- **Scenario:** A system uses the BLP model with the following classifications:
 - Files: Public, Confidential, Secret, Top Secret
 - Users:
 - **User A:** Clearance Confidential
 - **User B:** Clearance Secret
 - **User C:** Clearance Top Secret
- **Tasks:**
 5. Can User A read a Secret file? Why or why not?
 6. Can User B write data to a Confidential file? Explain.
 7. Can User C write a Top Secret file into a Secret directory? Why?

Exercise 2: Preventing Information Leakage

- **Scenario:** A hospital uses the BLP model to classify medical data:
 - Data Classifications: General, Confidential, Highly Confidential
 - Users:
 - **Doctor:** Clearance Confidential
 - **Nurse:** Clearance General
 - **Specialist Doctor:** Clearance Highly Confidential
- **Tasks:**
 8. Can the **Doctor** access Highly Confidential records? Why or why not?
 9. Can the **Specialist Doctor** share Highly Confidential data with the **Nurse**? Why or why not?

Summary

The **Bell-LaPadula Model** is ideal for environments requiring strict confidentiality, such as government or military systems. Its focus on **No Read Up** and **No Write Down** prevents data leaks across classification boundaries. However, it does not address integrity or other security concerns, making it insufficient for systems needing broader protections.

6. Role-Based Access Control (RBAC)

Definition:

Role-Based Access Control (RBAC) is a model where access to resources is granted based on the **roles** assigned to users within an organization. Permissions are associated with roles, not directly with users, allowing for scalable and manageable access control.

Key Concepts of RBAC

1. Role:

- A job function or responsibility within an organization (e.g., Manager, Engineer, HR Staff).
- Roles determine what actions a user can perform on resources.

2. Permission:

- Specific operations that can be performed on resources (e.g., read, write, delete).

3. User:

- A person or entity assigned one or more roles.

4. Session:

- A user's active session determines which role(s) they assume at a given time.

5. Hierarchy (Optional):

- Roles can be structured hierarchically, where higher roles inherit the permissions of lower roles.

Examples

Example 1: Employee Access in a Company

- **Scenario:**
 - Roles: Admin, Manager, Employee.
 - Permissions:
 - Admin: Full access to the system (read/write/delete).
 - Manager: Access to employee performance reports (read/write).
 - Employee: Access to their personal data (read-only).
- **Explanation:**
 - An Admin can perform all tasks, including adding or removing users.
 - A Manager can update performance reports but cannot delete them.
 - An Employee can view their information but cannot modify or delete it.

Example 2: University Access Control

- **Scenario:**
 - Roles: Professor, Student, Admin Staff.
 - Permissions:

- Professor: Grade assignments and access course materials (read/write).
- Student: Submit assignments and access course materials (read).
- Admin Staff: Manage course registration (read/write).
- **Explanation:**
 - A Professor can upload grades and course materials.
 - A Student can only view materials and submit their work.
 - Admin staff handle administrative tasks like enrollment.

Example 3: Cloud Service Management

- **Scenario:**
 - Roles: SuperAdmin, Developer, Viewer.
 - Permissions:
 - SuperAdmin: Full control of all resources (read/write/delete).
 - Developer: Deploy and manage code (read/write).
 - Viewer: View logs and performance metrics (read-only).
- **Explanation:**
 - The SuperAdmin manages all aspects of the cloud environment.
 - A Developer works on application deployment.
 - A Viewer can monitor the system without making changes.

Advantages of RBAC

1. **Scalability:** Adding users or resources requires updating roles, not individual permissions.
2. **Simplicity:** Centralized control over access based on roles.
3. **Security:** Minimizes unauthorized access by restricting permissions to roles.
4. **Auditability:** Easy to track who has access to what resources through roles.

Exercises

Exercise 1: Assigning Roles

- **Scenario:**

A hospital uses RBAC to manage access to medical records.

 - Roles: Doctor, Nurse, Admin.
 - Permissions:
 - Doctor: View and update patient records (read/write).
 - Nurse: View patient records (read-only).
 - Admin: Manage user accounts (read/write).
- **Tasks:**
 1. Assign the appropriate role to the following users:
 - Alice (responsible for updating patient diagnoses).

- Bob (responsible for viewing patient records but cannot edit them).
 - Carol (manages the hospital's user database).
2. Explain why a Nurse cannot edit patient diagnoses.

Exercise 2: Hierarchical RBAC

- **Scenario:**
An IT company has a hierarchy of roles:
 - Manager: Can approve budgets and view team reports.
 - Team Lead: Can view and edit team reports.
 - Employee: Can view team reports.
- **Tasks:**
 1. Determine if an Employee can edit team reports. Why or why not?
 2. Explain if a Manager can perform all the tasks of a Team Lead.

Exercise 3: Multi-Role Assignment

- **Scenario:**
A school uses RBAC for its library system:
 - Roles: Librarian, Teacher, Student.
 - Permissions:
 - Librarian: Add, remove, and update books (read/write/delete).
 - Teacher: Borrow books and reserve books (read/write).
 - Student: Borrow books (read).
- **Tasks:**
 1. Can a Teacher add a new book to the system? Why or why not?
 2. If a Librarian is also a Student, what permissions will they have?

Summary

Role-Based Access Control (RBAC) simplifies access control by associating permissions with roles rather than individual users. It is scalable, secure, and ideal for organizations with structured job functions. These examples and exercises illustrate how RBAC can be applied in real-world scenarios for effective resource management.

RBAC0 and RBAC1

RBAC0 and RBAC1 are part of the **NIST RBAC model** hierarchy, which defines different levels of Role-Based Access Control (RBAC) functionality:

1. **RBAC0 (Core RBAC)**: The basic RBAC model that includes roles, users, and permissions.
2. **RBAC1 (Hierarchical RBAC)**: Builds on RBAC0 by adding a **role hierarchy**, allowing roles to inherit permissions from other roles.

6.1. RBAC0 (Core RBAC)

Definition:

RBAC0 is the foundational model of RBAC. It provides:

- **Roles**: Define what actions can be performed on resources.
- **Users**: Assigned to one or more roles.
- **Permissions**: Associated with roles, determining access to resources.

Key Characteristics:

1. Roles are directly assigned to users.
2. Permissions are directly associated with roles.
3. No role inheritance or hierarchy.

Example of RBAC0

Scenario:

A hospital system has the following roles and permissions:

- - Roles:
 - Doctor: View and update patient records.
 - Nurse: View patient records.
 - Admin: Manage user accounts.
- Permissions:
 - View patient records → Assigned to Doctor and Nurse.
 - Update patient records → Assigned to Doctor.
 - Manage user accounts → Assigned to Admin.

Explanation: Users are directly assigned roles:

- Alice is a Doctor.
- Bob is a Nurse.
- Carol is an Admin.

No **role hierarchy exists**, so there's no inheritance between roles.

6.2. RBAC1 (Hierarchical RBAC)

Definition:

RBAC1 extends RBAC0 by introducing a **role hierarchy** where higher-level roles inherit permissions from lower-level roles.

Key Characteristics:

1. **Role Hierarchy:** Permissions of lower roles are inherited by higher roles.
2. Simplifies access control by reducing redundant assignments.

Example of RBAC1

Scenario:

Expanding the hospital system: Roles and hierarchy:

- Head Doctor (inherits Doctor permissions).
- Doctor (inherits Nurse permissions).
- Nurse.

Permissions:

- Nurse: View patient records.
- Doctor: View and update patient records.
- Head Doctor: All Doctor permissions, plus approve surgeries.

Explanation: Role hierarchy simplifies management:

- A Doctor automatically has all Nurse permissions.
- A Head Doctor automatically has all Doctor permissions, plus the ability to approve surgeries.

Users are assigned roles: -

- Alice is a Head Doctor.
- Bob is a Doctor.

- Carol is a Nurse.

Example of Role Hierachy:

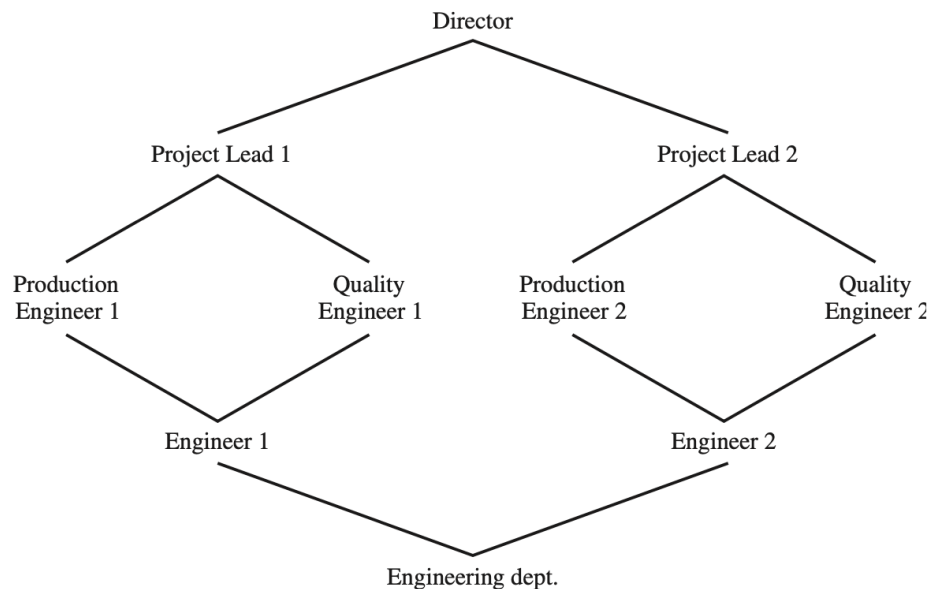


Figure 4.9 Example of Role Hierachy

Comparison of RBAC0 and RBAC1

Aspect	RBAC0	RBAC1
Hierarchy	No role hierarchy.	Includes role hierarchy for inheritance.
Permission Assignment	Directly to roles.	Inherited permissions from lower roles.
Management Complexity	Simple but may involve redundant permissions.	Reduces redundancy through inheritance.
Use Case	Small systems with fewer roles.	Large systems with hierarchical roles.

Exercises

Exercise 1: Assigning Permissions in RBAC0

- **Scenario:**
A company has the following roles and permissions:
 - Roles:
 - Employee: View company announcements.
 - Manager: Approve leave requests.
 - Admin: Add or remove users.
 - Permissions:

- View announcements → Assigned to Employee.
- Approve leave requests → Assigned to Manager.
- Manage users → Assigned to Admin.

Tasks:

1. Assign the role Manager to Alice. What permissions does she have?
2. Can Bob, who is an Employee, approve leave requests? Why or why not?

Exercise 2: Using Hierarchical RBAC1

- **Scenario:**

A school has the following roles and hierarchy:

- Principal (inherits all permissions from Teacher).
- Teacher (inherits all permissions from Assistant).
- Assistant.

Permissions: - Assistant: View student attendance records. - Teacher: View and grade assignments. - Principal: Manage school policies.

Tasks: 1. If Carol is a Principal, what permissions does she have? 2. If Bob is a Teacher, can he manage school policies? Why or why not? 3. Explain how role hierarchy simplifies access management in this system.

Summary

- **RBAC0** is the core RBAC model with direct assignment of roles and permissions.
- **RBAC1** adds a hierarchy, allowing roles to inherit permissions, which simplifies management in large systems.
- **Key takeaway:** RBAC1 is more efficient for organizations with a structured hierarchy, while RBAC0 is better suited for smaller, simpler setups.